**DOI: 10.1587/elex.22.20250219**
**Received: April 8, 2025**
**Accepted: May 7, 2025**
**Publicized: May 16, 2025**

IEICE Electronics Express, Vol.xx, No.x, xxxx

LETTER

# Design of hierarchical cache coherence protocol based on Chiplet architecture

**Jianghua Gui**[1,2,a)], **Bing Li**[1], **Tongtong Guo**[2], **Anzhou Lai**[2], **and Shuaishuai Zhang**[2]

**Abstract** This paper proposes a directory-based hierarchical cache coherence protocol for highly scalable Chiplet architectures. The proposed protocol can be seamlessly divided into two independent levels, the first level handling the inter-core cache coherency within a single Die while the second level dealing with the intra-Die cache coherency across multiple Dies. The proposed protocol is implemented using a two-level directory structure which exhibits superior scalability in terms of storage overhead. Simulation results indicate our approach using a two-level directory structure reduces the miss rate of the shared last-level cache (LLC) as compared to conventional approach using a single-level directory structure. This reduction enhances overall cache performance as the average memory access latency is reduced.
**key words:** cache coherence protocol, Chiplet architecture, MESI, hierarchical design
**Classification:** integrated circuits

## 1. Introduction

Chiplet architecture is considered to be a key technology to integrate more cores into Soc with lower overall cost and higher benefits, thereby continuing Moore's Law [1,2]. Despite the potential of the Chiplet architecture to enable multi-core systems to scale to many-core systems, maintaining cache coherence in such systems is more complex than in traditional multi-core systems [3].

One possible approach is utilizing a single-level directory, implementing reliable protocols such as MESI [4] to track and manage the shared states of cache lines among cores and multiple dies. However, the centralized directory structure exhibits low scalability due to the rapid growth of storage overhead as the number of cores increases [5]. Moreover, as the number of cores in the system increases, if only the traditional directory coherence protocol is used, it will not only consume on-chip resources, but also increase the on-chip network load and add indirection delay [6,7], diminishing the advantages of scaling multi-cores through Chiplet technology.

In this work, we propose a novel hierarchical protocol to address cache coherence in Chiplet architecture, which implies a reliable 2-level protocol and a 2-level directory structure. The main goal of the proposed protocol is to enable easy scaling from a single chiplet to many chiplets connecting to the second-level directory without reimplementation and testing of the protocol but simply extending the size of the sharing vector maintained in the directory. The proposed protocol is expected to improve the scalable performance of cache coherence management by reducing the directory storage overhead and increasing the shared cache hit rates.

The rest of the paper is organized as follows: some related published articles are summarized in section 2, the proposed hierarchical protocol and detailed storage overhead calculation are introduced in section 3, the method of forwarding data between dies is proposed in section 4, experimental results are discussed in section 5, finally, conclusions are described in section 6.

## 2. Related works

In monolithic multi-core processors, there is only one logically shared Last level cache (LLC), which is visible to all cores in the processor and shared system-wide[8-10] However, in Chiplet-based processors, referring to the designs of AMD EPYC[11,12], Intel Xeon Sapphire Rapids [13], and other processor, a single Compute Die accommodates a piece of LLC, which is only shared by the cores within it. The characteristic of the architecture has the natural advantage of multi-level management of cache coherence [14-16]. As paper [17] has revealed, the coherence within an AMD Compute Complex (AMD CCX) is handled locally by the LLC controller, the coherence across AMD CCX within a compute die is handled by AMD Infinity Fabric (IF)[18], while the IF is also responsible for managing cache coherence between Compute Dies.

Related research achievements on cache coherence protocols in Chiplet architecture focus on combining traditional ways: the snooping protocol and the directory protocol, to implement hierarchical cache coherence management. A novel hybrid cache coherence using

[1] School of Microelectronics, Southeast University, China
[2] China Electronics Technology Group Corporation No.58 Research Institute, China
[a)] 230199180@seu.edu.cn

emerging links is proposed in [19], enabling hierarchy by linking local directory for intra-Chiplet coherence and global snooping for coherence between Chiplets. The global emerging link exhibits high performance in reducing coherent overheads, including latency, energy consumption, and global directory storage. However, the design approach necessitates that the cache coherence design for SoCs not only address coherence issues but also tackle reliability problems at multiple levels associated with emerging interconnection methods. Relatively, the literature [20] proposes a hierarchical cache coherence scheme through an internal snooping bus within a die and global directory between dies. They assume that there are fewer cores within a die, thus, it's feasible to implemented an ordered bus.

## 3. Design of hierarchical cache coherence protocol based on Chiplet architecture

### 3.1 Cache hierarchy in Chiplet architecture

The characteristics of a typical Chiplet architecture processor are shown in Fig. 1 [21].

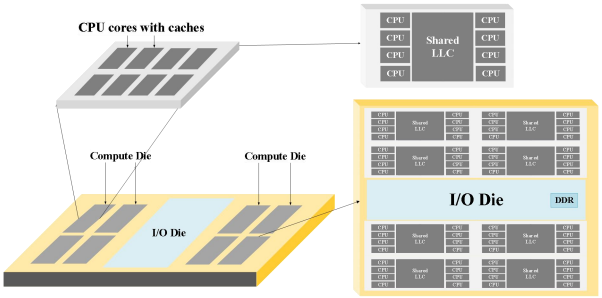


**Fig. 1** Typical Chiplet architecture

In Fig. 1, the main memory is not mounted on a single Compute Die in distribution, but the memory access is initiated by the DDR controller on the I/O Die. From the perspective of the Compute Die, all memory address spaces can be accessed through the memory control unit specified on the I/O Die without accessing across the Compute Dies, balancing the NUMA (Non-Uniform Memory Access) effect [22-24].

In a Compute Die, each core has its own private L1 cache and L2 cache, and shares the last level cache (LLC) in common. All Compute Dies share the main memory through interconnection on the Soc.

### 3.2 Two-level cache coherence protocol design

Assuming that the Chiplet architecture processor divides N cores into n Compute Dies × k cores/Compute Die as shown in Fig. 2.

In our proposal, we incorporate a 1st-level protocol managing data sharing among all cores that share the local LLC. Additionally, a 2nd-level protocol is implemented to manage data sharing among multiple Compute Dies. The fundamental assumption underlying this design is that the LLC within a Die is inclusive of the private caches within local Die [25,26].

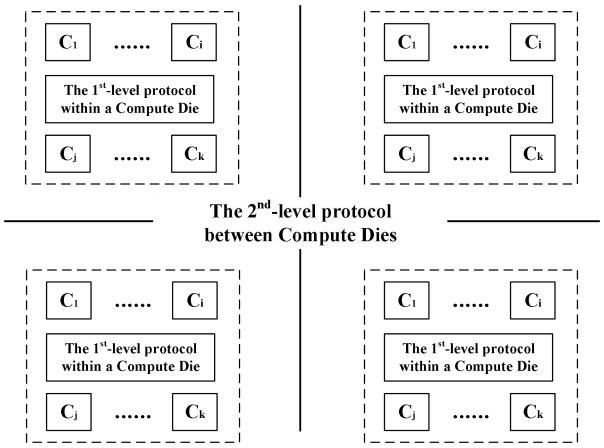


**Fig. 2** Design overview of the 2-level cache coherence protocol

The hierarchical protocol is designed based on the traditional MESI protocol and we optimize it to tune the Chiplet-based architecture. Firstly, given that the transmission latency of the interconnect networks between Compute Dies in the system is significantly greater than that within in a Die, we grant the 1st -level protocol within the Die full authority to manage cache coherence locally. This means, when only the local Die possesses a cached copy of a data on a certain address, the 1st-level protocol can manage coherence entirely without the need to notify any external peers.

Meanwhile, when a copy of data exists in the local Compute Die and other Dies, the cores within the local Die can still read valid data. However, under the constraints of the write-invalidate policy [27], write requests must invalidate the copies both in other cores' private caches and in other Dies before completed.

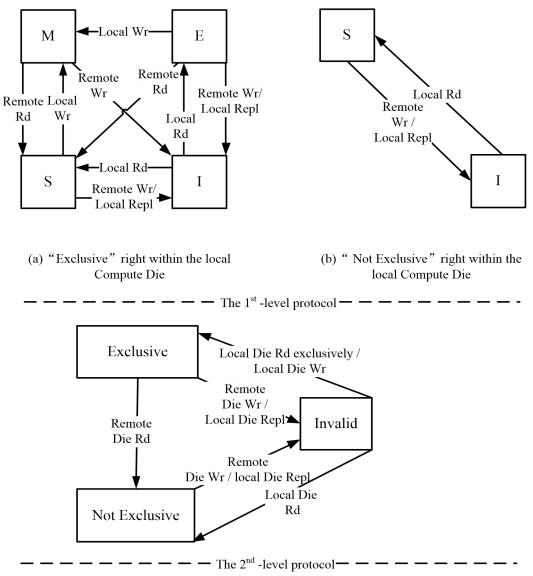


**Fig. 3** Two-level cache coherence protocol model

Therefore, the two-level hierarchical protocol model proposed in this work is shown in Fig.3， the “Rd” represents “Read” requests, the “Wr” represents “Write” and the “Repl” represents “Replacement”.

Directory to implement the two–level protocol is also designed to be a hierarchical structure as shown in Fig.4. To limit the directory storage overhead. The 1st-level directory is designed to as “In-Cache” directory, which contains 1st-level coherent information for data copies cached by the local Compute Die. The 2nd-level directory, in principle, contains the 2nd-level coherent information for all cache lines within system. Therefore, once a valid cache line data is caches in cache nodes, it will be signed with two types of coherent states, namely, the “right” state between dies and the “state” state within the local die.

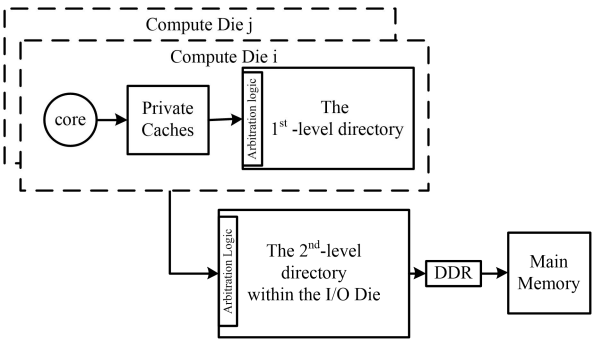


**Fig. 4** Two-level cache coherence directory model

Global coherence state storage mark is shown in Table I. The table shows that it only costs 1 bit in length to storage the 2nd-level coherent state in 2nd-level directory.

**Table I** Global cache coherence state description in the two-level protocol and the two-level directory

| line valid | “right” between dies | Storage mark to sign “right” between dies | “state” within a die | Storage mark to sign “state” within a die |
|---|---|---|---|---|
| Y | Exclusive | 1 | M | 11 |
| | | | E | 10 |
| | | | S | 01 |
| | | | I | 00 |
| | Not Exclusive | 0 | S | 01 |
| | | | I | 00 |
| N | Invalid | -- | -- | -- |

The hierarchical design proposed in this paper also demonstrates modular advantages. As it’s shown in Fig. 5, we have designed the two-level protocol and directory separately, allowing for the expansion of the number of Dies within the system by only replacing the 2nd-level directory on the IOD accordingly, without the need to redesign the cache coherence protocol or any coherence directory structure within the Dies. Furthermore, we have supported rapid iteration of processor products from the perspective of cache coherence.

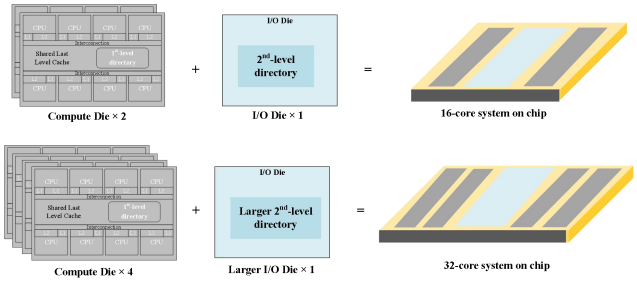


**Fig. 5** The method to expand the Chiplet-based system by the 2-level cache coherence design

The proposal including the two-level protocol and the two–level directory splits the marks recording the sharer bit vectors in the traditional directory into two levels. Assuming the physical address of data takes 64 bits, and a cacheline takes 64 Bytes in length. Since the directory traces the blocks cached in the system, assuming that the capacity of the LLC is 2MB/Core, the storage overhead of the single-level directory is shown as Eq. (1), the unit is “Kb”:

$$\begin{aligned} M_1 &= \frac{M_{LLC}}{len(cacheline)} M_{entry} \\ &= \frac{N \times 2MB}{64B}(1+2+64-\log_2 64+N) \\ &= 32N(61+N) \end{aligned} \tag{1}$$

In our proposal, the storage overhead of the –level directory can be calculated as Eq. (2), the “k” represents the number of cores per Compute Die and the “n” represents the number of Compute Dies within system and the unit is “Kb”. Therefore, N is the product of “n” and “k”.

$$\begin{aligned} M_2 &= nM_{1st} + M_{2nd} \\ &= n[\frac{k \times 2MB}{64B}(1+2+k)] + \frac{N \times 2MB}{64B}(1+1+64-\log_2 64+n) \\ &= 32N(63+k+n) \end{aligned} \tag{2}$$

The Eq. (2) indicates that, once the number of cores within a Compute Die stays the same, the complexity of the storage overhead is proportional to $n^2$ rather than $N^2$.

The calculated result of the Eq. (2) has a lower bound when “n” equals “k”, specific results are shown in Table II and the result of $M_2$ are described in its lower bound.

**Table II** Comparison of storage overhead between single-level directory and the two-level directory

| Number of cores | $M_1$(MB) | $M_2$(MB) | Saving percentage |
|---|---|---|---|
| 16 | 4.8125 | 4.4375 | 7.79% |
| 32 | 11.625 | 9.375 | 19.35% |
| 64 | 31.25 | 19.75 | 36.80% |
| 128 | 94.5 | 43.5 | 53.97% |
| 256 | 317 | 95 | 80.62% |
| 512 | 1146 | 222 | 88.29% |

The comparison of the results in Table II shows that the design of the two-level directory effectively reduces the directory storage overhead comparing with the traditional single-level directory, and as the number of cores increases, the percentage of saved storage overhead

increases.

## 4. The data forwarding to accelerate data transferring for loading requests

In this work, we also look for ways to improve the performance of the system through the coherence scheme proposed above.

During the pipeline execution of a single core, storing requests often do not affect the execution once missed in cache systems because of the write buffer. However, for loading requests, the absence of the operand may cause the pipeline to stall when missing in the cache system. Therefore, the loading request is on the critical path of memory access [28].

It can be known that when a cache miss occurs for the data of a certain target address within a single Compute Die, under the constraints of the coherence protocol states, a read miss request needs to access the 2nd-level directory on the IOD, which will initiate a memory access request if there is no link for transferring cache data between Dies to achieve simpler inter-chip interconnection. Therefore, as shown in the Fig.6. In the case where the target address data is shared between Dies, it is still necessary to request data from the memory.

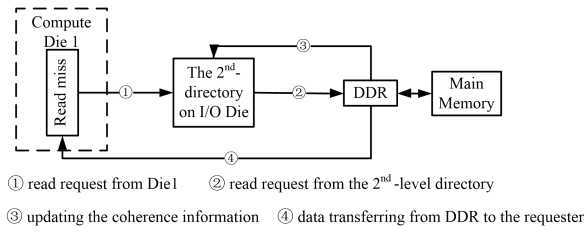


**Fig. 6** Traditional data transferring method for loading requests that miss in Compute Dies

We propose to reuse the valid copies in the system, as shown in Fig.7 compared with the plan in Fig.6 considering the long latency of data transmission from the main memory to the requester.

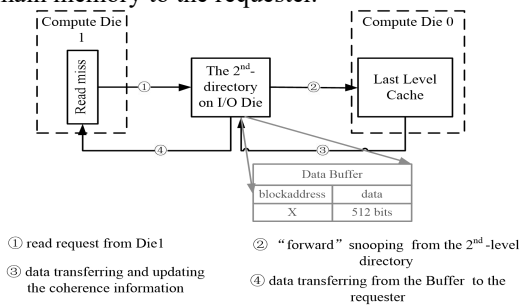


**Fig. 7** Data forwarding scheme for loading requests that miss in Compute Dies

As shown in Fig.7, the proposal only works when a sharer's Die containing a valid and clean copy to forward data in order to optimize the memory access latency. For the "Read miss" request from Die 1, the coherence information of the target address X can firstly be obtained after searching the 2nd-level directory. Since the "Not-exclusive" right allows multiple sharers in the sharer bit vector recorded in the directory, the sharer with the smallest ID will be selected as the proxy forwarder, the formula of choosing destination is shown in Eq. (3).

$$dest\_id = n \,\&\, \overline{(n-1)} \tag{3}$$

Once the selected forwarder responds to the absence of data confirmation, since the copy in "Shared" state may be silently replaced without notifying the directory, the 2nd -level directory will initiate a request to access memory again, and the process will return to that shown in Fig.6.

The overhead of the proposed data forwarding mechanism includes two main aspects: 1) Adding a new "forward" snooping and corresponding acknowledgment in original snooping transactions of MESI Protocol. The snooping results in lower implementation cost because it reduces remote private cache access hops by only involving LLC replication. 2)Implementing a Data Buffer on IOD to temporarily store acknowledged data during directory updates and transfers. Buffer usage peaks under concurrent requests possibly, and its capacity must scale with the 2nd-level directory's pipelined processing parallelism.

## 5. Experimental results and analysis

This section conducts performance testing experiments based on a cache coherence simulation platform and compares our proposal with the traditional MESI protocol and single-level directory.

### 5.1 Experimental configuration

We support the OpenMP (Open Multiple Parallel) [29] programming model on the simulation platform, which is the most widely used type of parallel programming. Two programs in the SPEC OMP 2012 suite (nab_md and bots-sparselu) and four programs in the PARSEC 3.0 suite (ocean, blackscholes, fluidannimate) were selected for testing. Specific configurations are shown in Table III.

**Table III** Experimental basic parameter configuration

| Parameter | Parameter value setting |
|---|---|
| Instruction Set | RISC-V |
| L1 Cache | 16KB, 4-way, 2 CPU Cycle |
| L2 Cache | 64KB, 8-way, 10 CPU Cycle |
| LLC | 1MB/Core, 16-way, 50 CPU Cycle |
| The 2nd-level directory | No limitation of capacity in simulation, 80 CPU Cycle |
| Memory | Capacity: Unlimited<br>DDR Channels: Same as Die quantity<br>Average memory access latency: 288 CPU Cycle |
| Related cache strategies | Write Protocol: Write Back<br>Replacement policy: LRU |
| Cacheline length | 64 Bytes |
| Chiplet architecture | Total number of cores in the system: N (N = 16)<br>Number of compute Dies: k（configurable）<br>Number of Cores within a Die: n（configurable） |

### 5.2 Performance Indicators

The Average Memory Access Time is the most important indicator to measure the performance of the cache system. The calculation formula is shown in Eq. (4).

$$AMAT = Hit\_time_{L1} + Miss\_rate_{L1}(Hit\_time_{L2} + Miss\_rate_{L2}( Hit\_time_{LLC} + Miss\_rate_{LLC} \times Latency_{Mem})) \quad (4)$$

Since we assign the cachelines with two-level of states, the data copy in the states of "Exclusive right" and "Invalid state" in the system will still stay in the shared cache and remain valid. The read and write requests will still result in hits on these copies. Therefore, compared with the traditional MESI protocol, the miss rate of LLC will be reduced. Miss rate of LLC can be calculated by Eq. (5).

$$Miss\_rate_{LLC} = \frac{LLC_{miss}}{LLC_{req}} \quad (5)$$

### 5.3 Experimental results and analysis

Fig.8 and Fig.9 show the statistic results of the miss_rate of LLC in tested applications. And Table IV shows the comparison results of corresponding difference of AMAT between the single-level cache coherence protocol and our proposal.

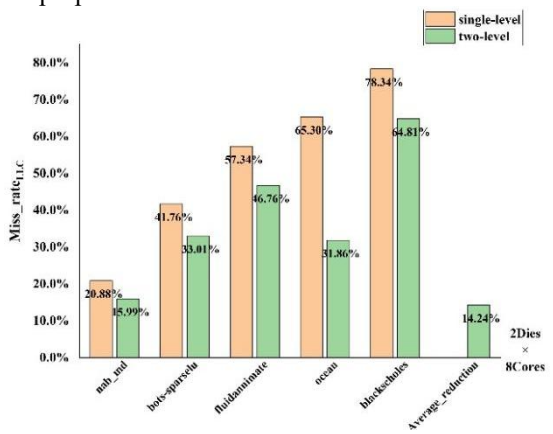


**Fig.8** Comparison of $Miss\_rate_{LLC}$ between single-level cache coherence protocol and our proposal, system scale:2 Dies ×8 Cores

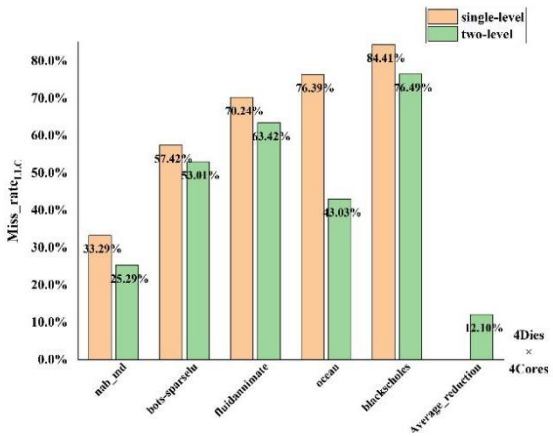


**Fig.9** Comparison of $Miss\_rate_{LLC}$ between single-level cache coherence protocol and our proposal, system scale:4 Dies ×4 Cores

**Table IV** Comparison results of the $\Delta AMAT$

| Testing Procedure | System scale | $\triangle AMAT$(%) | *average*(%) |
|---|---|---|---|
| nab_md | 2×8 | 10.05 | 11.59 |
| | 4×4 | 13.12 | |
| bots-sparselu | 2×8 | 12.59 | 8.89 |
| | 4×4 | 5.18 | |
| fluidannimate | 2×8 | 12.42 | 9.69 |
| | 4×4 | 6.96 | |
| ocean | 2×8 | 35.92 | 33.98 |
| | 4×4 | 32.03 | |
| blackscholes | 2×8 | 12.74 | 9.90 |
| | 4×4 | 7.05 | |

First of all, the comparison results between Fig. 8 and Fig. 9 show that for processors based on the Chiplet architecture, adopting different system scales will lead to different hit rates of parallel programs in the shared cache. Basically, integrating more cores within a die and restricting more data sharing behaviors within the same die without the need for cross-die access can result in a shorter average memory access latency, resulting that the cache miss rate of the same program under the management of the two types of cache coherence protocols in Fig. 9 being higher than that in Fig. 8.

Secondly, the two-level protocol exhibits a lower cache miss rate and shortens the average memory access latency within the Chiplet system. In the experiments conducted on the same system, the higher the hit rate of the shared cache, the faster the task acceleration. For example, in the ocean program, the average miss rate is reduced by up to 33.40%, which in turn significantly shortens the average memory access time by up to 33.98%. The performance of other applications differs from that of ocean because the scales of their working data sets are different. Qualitatively, when the shared cache on the hardware has the advantages of large capacity and short latency, the possibility of hitting the shared cache is greater and it's less relative with the application program but more relative with the size of the working set, because the data in the shared cache itself is equivalent to an "agent of the memory".

Additionally, under the proposal, the shared cache also has the management authority for the cache coherence activities within the die, eliminating the need for cross-die access. Thus, compared with the centralized directory between dies, it reduces the high memory access latency caused by cache coherence management and improves the performance of the cache system.

## 6. Conclusion

This paper proposes a directory-based hierarchical cache coherence protocol designed for Chiplet architectures. Simulation results reveal that, as compared to MESI protocol, the proposal retains valid yet temporarily non-shared dirty data in the shared LLC rather than writing it back to main memory. The strategy effectively lowers LLC miss rates, thereby reducing average memory access time and enhancing system performance.

A notable advantage of this protocol lies in its modular design. When homogeneous dies or recycled IPs [30] are integrated into the system, only the global directory requires replacement. This "plug-and-play" [31] capability manifests exceptional system modularity and scalability while simplifying maintenance while preserving core functionalities.

**References**


[1] W. Tang, *et al.*: "Energy-Efficient Parallel Interconnects for Chiplet Integration," IEEE Micro **45** (2024) 41 (DOI: 10.1109/MM.2024.3450841).

[2] R. Munoz: "Furthering Moore's Law Integration Benefits in the Chiplet Era," IEEE Design & Test **41** (2024) 81 (DOI: 10.1109/MDAT.2023.3302809).

[3] C. Li, *et al.*: "Accelerating Cache Coherence in Manycore Processor through Silicon Photonic Chiplet," 2022 IEEE International Conference on Computer-Aided Design (ICCAD) (2022) 1 (DOI: 10.1145/3508352.354933).

[4] K. Rojaran, *et al.*: "Revisiting the Complexity of Hardware Cache Coherence and Some Implications," ACM Transactions on Architecture and Code Optimization **11** (2015) 37 (DOI: 10.1145/2663345).

[5] R. Titos-Gil, *et al.*: "Way Combination for an Adaptive and Scalable Coherence Directory," IEEE Transactions on Parallel and Distributed Systems **30** (2019) 2608 (DOI: 10.1109/TPDS.2019.2917185).

[6] S. H. Gade, *et al.*: "A Novel Hybrid Cache Coherence with Global Snooping for Many-core Architectures," ACM Transactions on Design Automation of Electronic Systems **27** (2022) 1 (DOI: 10.1145/3462775).

[7] L. Huang, *et al.*: "Integrated coherence prediction: Towards efficient cache coherence on noc-based multicore architectures," ACM Transactions on Design Automation of Electronic Systems **19** (2014) 24 (DOI: 10.1145/2611756).

[8] Y. Wang, *et al.*: "Research on the Management Strategy of the Last Level Cache Sharing Multi-Core Processor," International Journal of Grid & Distributed Computing **8** (2015) 287 (DOI: 10.14257/ijgdc.2015.8.5.29).

[9] J. Chang, *et al.*: "The 65-nm 16-MB Shared On-Die L3 Cache for the Dual-Core Intel Xeon Processor 7100 Series," IEEE Journal of Solid-State Circuits **42** (2007) 846 (DOI: 10.1109/JSSC.2007.892185).

[10] S. Fide, *et al.*: "Proactive Use of Shared L3 Caches to Enhance Cache Communications in Multi-Core Processors," IEEE Computer Architecture Letters **7** (2008) 57 (DOI: 10.1109/L-CA.2008.10).

[11] M. Huang, *et al.*: "An Energy Efficient 32-nm 20-MB Shared On -Die L3 Cache for Intel® Xeon® Processor E5 Family," IEEE Journal of Solid-State Circuits **48** (2013) 1954 (DOI: 10.1109/JSSC.2013.2258815).

[12] S. Naffziger, *et al.*: "Pioneering Chiplet Technology and Design for the AMD EPYC™ and Ryzen™ Processor Families: Industrial Product," 2021 IEEE Annual International Symposium on Computer Architecture (ISCA) (2021) 57 (DOI: 10.1109/ISCA52012.2021.00014).

[13] M. Mattioli: "Rome to Milan, AMD Continues Its Tour of Italy," IEEE Micro **41** (2021) 78 (DOI: 10.1109/MM.2021.3086541).

[14] N. Nassif, *et al.*: "Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor," 2022 IEEE International Solid-State Circuits Conference (ISSCC) (2022) 44 (DOI: 10.1109/ISSCC42614.2022.9731107).

[15] M. Velten, *et al.*: "Memory Performance of AMD EPYC Rome and Intel Cascade Lake SP Server Processors," 2022 ACM/SPEC International Conference on Performance Engineering (ICPE) (2022) 165 (DOI: 10.1145/3489525.3511689).

[16] G. Katevenis, *et al.*: "Impact of Cache Coherence on the Performance of Shared-Memory based MPI Primitives: A Case Study for Broadcast on Intel Xeon Scalable Processors," 2023 the 52nd International Conference on Parallel Processing (ICPP) (2023) 295 (DOI: 10.1145/3605573.3605616).

[17] R. Bhargava, *et al.*: "AMD Next-Generation 'Zen 4' Core and 4th Gen AMD EPYC Server CPUs," IEEE Micro **44** (2024) 8 (DOI: 10.1109/MM.2024.3375070).

[18] T. Wang, *et al.*: "Application Defined On-chip Networks for Heterogeneous Chiplets: An Implementation Perspective," 2022 IEEE Symposium on High-Performance Computer Architecture (HPCA) (2022) 1198 (DOI: 10.1109/HPCA53966.2022.00091).

[19] S. H. Gade, *et al.*: "Scalable Hybrid Cache Coherence Using Emerging Links for Chiplet Architectures," 2022 International Conference on VLSI Design (VLSID) (2022) 92 (DOI: 10.1109/VLSID2022.2022.00029).

[20] R. Wang, *et al.*: "Design of an efficient hybrid cache coherence protocol on Chiplet architecture," 2024 3rd International Conference on Algorithms, Microchips and Network Applications (AMNA) (2024) 131711 (DOI: 10.1117/12.3031958).

[21] S. Naffziger, *et al.*: "2.2 AMD Chiplet Architecture for High-Performance Server and Desktop Products," 2020 IEEE International Solid-State Circuits Conference (ISSCC) (2020) 44 (DOI: 10.1109/ISSCC19947.2020.9063103).

[22] L. N. Bhuyan, *et al.*: "Impact of CC-NUMA memory management policies on the application performance of multistage switching networks," IEEE Transactions on Parallel and Distributed Systems **11** (2000) 230 (DOI: 10.1109/71.841740).

[23] J. D. Booth, *et al.*: "A NUMA-Aware Version of an Adaptive Self-Scheduling Loop Scheduler," ACM Transactions on Architecture and Code Optimization **21** (2024) 4 (DOI: 10.1145/3680549).

[24] H. Nicholson, *et al.*: "Chaosity: Understanding Contemporary NUMA-Architectures," 2023 15th TPC Technology Conference on Performance Evaluation & Benchmarking (TPCTC) (2023) 59 (DOI: 10.1007/978-3-031-68031-1_5).

[25] M. M. K, *et al.*: "Why on-chip cache coherence is here to stay," Communications of the ACM **55** (2012) 78 (DOI: 10.1145/2209249.2209269).

[26] S. M. Khan, *et al.*: "Temporal-Based Multilevel Correlating Inclusive Cache Replacement," ACM Transactions on Architecture and Code Optimization **10** (2013) 33 (DOI: 10.1145/2541228.2555290).

[27] H. Grahn, *et al.*: "Evaluation of a Competitive-Update Cache Coherence Protocol with Migratory Data Detection," Journal of Parallel and Distributed Computing **39** (1996) 168: (DOI: 10.1006/jpdc.1996.0164).

[28] T. Li, *et al.*: "ADir/sub p/NB: a cost-effective way to implement full map directory-based cache coherence protocols," IEEE Transactions on Computers **50** (2001) 921 (DOI: 10.1109/12.954507).

[29] A. V. Martínez: "Parallelization of Array Method with Hybrid Programming: OpenMP and MPI," Applied Sciences **12** (2022) 7706 (DOI: 10.3390/app12157706).

[30] A. Huynh, *et al.*: "UCIe Standard: Enhancing Die-to-Die Connectivity in Modern Packaging," IEEE Micro **45** (2025) 26 (DOI: 10.1109/MM.2025.3526048).

[31] Taballione: "A Plug-and-Play Universal Photonic Processor for Quantum Information Processing," 2021 IEEE Hot Chips Symposium (HCS) (2021) 1 (DOI: 10.1109/HCS52781.2021.9566977).